

Surprise, Entropy & the Price of Being Wrong

Information theory, entropy, and the Kullback–Leibler divergence

From Data to Decisions, Week 1

ETH Zurich, Summer 2026



A guessing game

TH_ QU_CK BR_WN F_X

A guessing game

TH_ QU_ CK BR_ WN F_ X

- ▶ You filled the blanks instantly $\Rightarrow$ English is **predictable**
- ▶ Predictable $\Rightarrow$ **compressible**

Which of these would make a **good headline**?

"Sunny in the Sahara tomorrow"

vs.

"Snow in the Sahara tomorrow"

A guessing game

TH_ QU_ CK BR_ WN F_ X

- ▶ You filled the blanks instantly $\Rightarrow$ English is **predictable**
- ▶ Predictable $\Rightarrow$ **compressible**

Which of these would make a **good headline**?

“Sunny in the Sahara tomorrow”

vs.

“**Snow** in the Sahara tomorrow”

Information = **surprise**, and surprise is tied to **low probability**.

Designing a measure of surprise

We want a function $I(x)$ = “information carried by event x ”.

Three common-sense requirements:

Designing a measure of surprise

We want a function $I(x)$ = “information carried by event x ”.

Three common-sense requirements:

1. $I(x)$ depends only on the probability $p(x)$; rarer $\Rightarrow$ more information

Designing a measure of surprise

We want a function $I(x)$ = “information carried by event x ”.

Three common-sense requirements:

1. $I(x)$ depends only on the probability $p(x)$; **rarer $\Rightarrow$ more information**
2. Certainty is not news: $p(x) = 1 \Rightarrow I(x) = 0$

Designing a measure of surprise

We want a function $I(x)$ = “information carried by event x ”.

Three common-sense requirements:

1. $I(x)$ depends only on the probability $p(x)$; **rarer** $\Rightarrow$ **more information**
2. Certainty is not news: $p(x) = 1 \Rightarrow I(x) = 0$
3. Independent events **add**: $I(x \text{ and } y) = I(x) + I(y)$ but probabilities *multiply*:
 $p(x)p(y)$

Designing a measure of surprise

We want a function $I(x)$ = “information carried by event x ”.

Three common-sense requirements:

1. $I(x)$ depends only on the probability $p(x)$; **rarer** $\Rightarrow$ **more information**
2. Certainty is not news: $p(x) = 1 \Rightarrow I(x) = 0$
3. Independent events **add**: $I(x \text{ and } y) = I(x) + I(y)$ but probabilities *multiply*:
 $p(x)p(y)$

products



sums

the logarithm

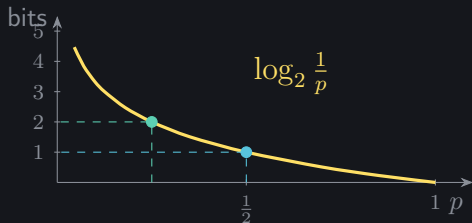
Surprisal

$$I(x) = -\log_2 p(x) = \log_2 \frac{1}{p(x)} \quad \text{measured in bits}$$

same formula, two costumes: "log of one-over- p " makes it obvious that surprisal is positive, and grows as p shrinks

event	p	bits
fair coin: heads	$1/2$	1
1 of 4 outcomes	$1/4$	2
lottery win	10^{-7}	≈ 23

One bit = one yes/no question
= one halving of uncertainty.



The Waiter's Code

A waiter signals orders to the kitchen in [binary](#), and dishes are not equally popular. Short codewords for frequent dishes, long ones for rare dishes. Let's play it.

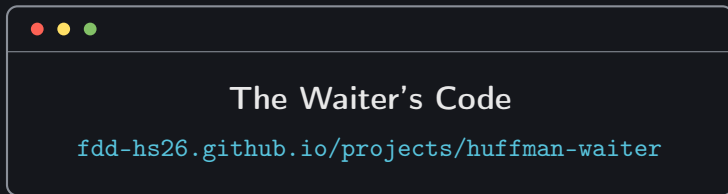
A rectangular window with rounded corners and a thin border. At the top left, there are three small circles in red, yellow, and green. The window contains the title 'The Waiter's Code' and a URL below it.

The Waiter's Code

fdd-hs26.github.io/projects/huffman-waiter

The Waiter's Code

A waiter signals orders to the kitchen in **binary**, and dishes are not equally popular. Short codewords for frequent dishes, long ones for rare dishes. Let's play it.



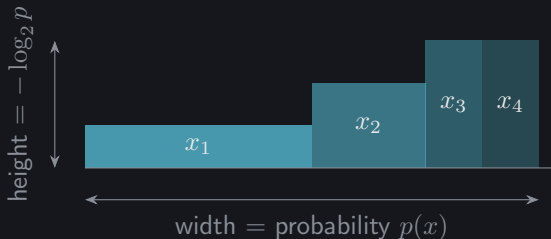
Two sentences to take away:

- ▶ In an **optimal code**, each symbol costs exactly its **surprisal** $-\log_2 p$
- ▶ “Surprise” is **not a metaphor**; it is a number of **bits**

Entropy: average surprise

$$H(p) = \sum_x p(x) (-\log_2 p(x))$$

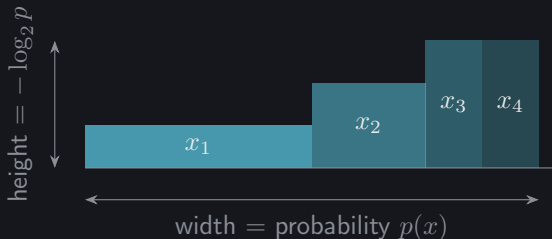
= how often $\times$ how surprising



total shaded area
= $H(p) = 1.75$ bits

Entropy: average surprise

$$H(p) = \sum_x p(x) (-\log_2 p(x)) \quad = \text{how often} \times \text{how surprising}$$



total shaded area
 $= H(p) = 1.75$ bits

- ▶ **Shannon:** no code can beat $H(p)$ bits/symbol on average: the floor of compression
- ▶ A property of the *source* itself, not of any particular code

Entropy is a measure of uncertainty

always sunny



coin (0.9, 0.1)



fair coin



→ H (bits)

- ▶ Deterministic: $H = 0$ nothing surprises you biased: $H \approx 0.47$ fair: $H = 1$
max for 2 outcomes
- ▶ **English:** uniform letters $\rightarrow \log_2 27 \approx 4.75$ bits/char; real English ≈ 1 bit/char $\Rightarrow$
 $\sim 80\%$ redundant (the guessing game!)

Entropy is a measure of uncertainty



- ▶ Deterministic: $H = 0$ nothing surprises you biased: $H \approx 0.47$ fair: $H = 1$
max for 2 outcomes
- ▶ **English:** uniform letters $\rightarrow \log_2 27 \approx 4.75$ bits/char; real English ≈ 1 bit/char $\Rightarrow$
 $\sim 80\%$ **redundant** (the guessing game!)

Feels wrong? If the biased coin seems *more* surprising, it's because you *assumed* coins are fair. That assumption is a **belief**, and surprise measured against a belief (rather than reality) is a different, very important quantity.

Hold that thought.

Back to the Waiter's Code

So far, each event carried information *about itself*.

New question: **how much does one observation tell you about another?**



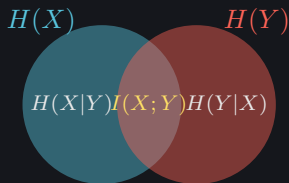
Watch for it in the game: knowing one thing (a regular customer, the previous order, the time of day) *shrinks your uncertainty* about what comes next.

Mutual information: what one thing tells us about another

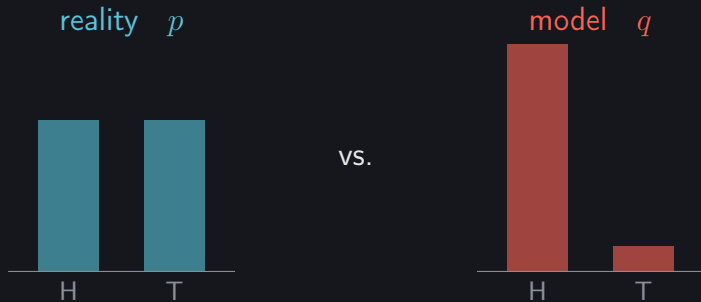
An email contains the word “lottery”. Suddenly you are far less uncertain whether it’s spam. The *word* carried information about the *label*.

$$I(X;Y) = H(X) - H(X | Y)$$

- ▶ The **drop in uncertainty** about X when you learn Y ($H(X|Y)$: what’s still unknown)
- ▶ Independent: $I = 0$; perfect copy: $I = H(X)$
- ▶ **Symmetric**: the **overlap** belongs to the pair; X tells you exactly as much about Y as Y about X

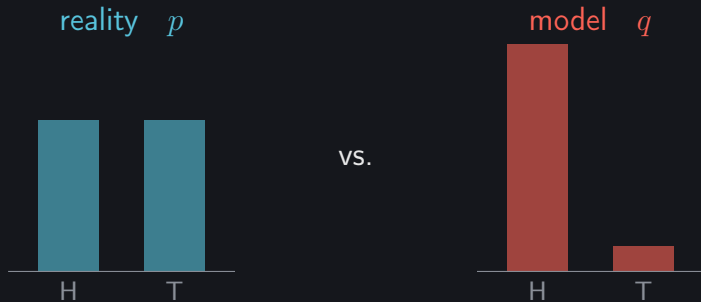


The pivot: two distributions



- ▶ p = how the world *actually* behaves
- ▶ q = what an agent *believes*: its internal model (like assuming the coin was fair!)

The pivot: two distributions



- ▶ p = how the world *actually* behaves
- ▶ q = what an agent *believes*: its internal model (like assuming the coin was fair!)

You are surprised according to q , but events arrive according to p .

Cross-entropy: surprise under the wrong model

$$H(p, q) = \sum_x p(x) (-\log_2 q(x))$$



p : how often $-\log_2 q$: how surprising

$= H(p, q)$

Two equivalent readings:

- ▶ **Belief reading**: average surprise of an agent living with model q in world p
- ▶ **Code reading**: average message length, when the code was designed for q but data arrives from p

The Kullback–Leibler divergence

The excess surprise deserves its own name:

$$D_{\text{KL}}(p \parallel q) = H(p, q) - H(p) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}$$

Three equivalent readings:

- ▶ the **extra surprise** per event, caused by believing q when the truth is p
- ▶ the **wasted bits** per symbol, using a code built for the wrong distribution
- ▶ a measure of **how far** the model q is from reality p

The Kullback–Leibler divergence

The excess surprise deserves its own name:

$$D_{\text{KL}}(\textcolor{teal}{p} \parallel \textcolor{red}{q}) = H(p, q) - H(p) = \sum_x p(x) \log_2 \frac{\textcolor{teal}{p}(x)}{\textcolor{red}{q}(x)}$$

Three equivalent readings:

- ▶ the **extra surprise** per event, caused by believing q when the truth is p
- ▶ the **wasted bits** per symbol, using a code built for the wrong distribution
- ▶ a measure of **how far** the model q is from reality p

A nice connection: mutual information is a KL divergence in disguise,
 $I(X; Y) = D_{\text{KL}}(p(x, y) \parallel p(x)p(y))$: how far the pair is from *pretend-independence*.

One picture to remember



Properties worth saying out loud:

- ▶ $D_{\text{KL}}(p \parallel q) \geq 0$ always
- ▶ $D_{\text{KL}}(p \parallel q) = 0$ exactly when $p = q$ a perfect model is never *extra*-surprised

A divergence, not a distance

reality **fair**, model **biased**



$$D_{\text{KL}} \approx 0.74 \text{ bits}$$

reality **biased**, model **fair**



$$D_{\text{KL}} \approx 0.53 \text{ bits}$$

Same pair of distributions, **different divergence**: one says how often events **occur**, the other says how **surprised** you are when they do.

Extreme case: $q(x) = 0$ but x happens $\Rightarrow D_{\text{KL}} = \infty$.

Being **certain and wrong** is infinitely costly; being vague is not.

Why machine learning cares

Training a model means making its beliefs q_θ match reality p .

The principled objective: minimize $D_{\text{KL}}(p \parallel q_\theta)$ make the price of being wrong as small as possible

But you will never see D_{KL} written in a training script: computing it requires knowing p , and nobody hands us the true distribution of text.

Why machine learning cares

Training a model means making its beliefs q_θ match reality p .

The principled objective: minimize $D_{\text{KL}}(p \parallel q_\theta)$ make the price of being wrong as small as possible

But you will never see D_{KL} written in a training script: computing it requires knowing p , and nobody hands us the true distribution of text. What the code *can* measure: the model's **surprise** on real samples.



this is the **loss function** in the code: $\mathcal{L}(\theta) = -\log_2 q_\theta(\text{actual next token})$;
averaged over real data it is exactly **cross-entropy** $H(p, q_\theta)$ (samples only, no explicit p needed)

Why the loss function is cross-entropy, not KL

$$\underbrace{D_{\text{KL}}(p \parallel q_\theta)}_{\text{what we want small}} = \underbrace{H(p, q_\theta)}_{\text{the loss function we can compute}} - \underbrace{H(p)}_{\text{a constant of the world}}$$

Why is $H(p)$ a **constant**?

- ▶ $H(p)$ depends only on p : the statistics of real data (real text, real coins)
- ▶ Training moves θ , and θ lives only inside q_θ ; nothing you do to the model changes **reality**
- ▶ So the two objectives differ by a number that **never moves**, and taking derivatives:

$$\nabla_\theta D_{\text{KL}}(p \parallel q_\theta) = \nabla_\theta H(p, q_\theta) - \underbrace{\nabla_\theta H(p)}_{=0} = \nabla_\theta H(p, q_\theta)$$

Why the loss function is cross-entropy, not KL

$$\underbrace{D_{\text{KL}}(p \parallel q_\theta)}_{\text{what we want small}} = \underbrace{H(p, q_\theta)}_{\text{the loss function we can compute}} - \underbrace{H(p)}_{\text{a constant of the world}}$$

Why is $H(p)$ a **constant**?

- ▶ $H(p)$ depends only on p : the statistics of real data (real text, real coins)
- ▶ Training moves θ , and θ lives only inside q_θ ; nothing you do to the model changes **reality**
- ▶ So the two objectives differ by a number that **never moves**, and taking derivatives:

$$\nabla_\theta D_{\text{KL}}(p \parallel q_\theta) = \nabla_\theta H(p, q_\theta) - \underbrace{\nabla_\theta H(p)}_{=0} = \nabla_\theta H(p, q_\theta)$$

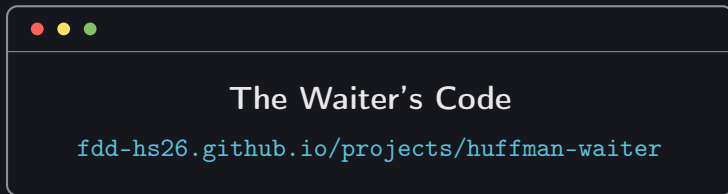
Same derivatives, same minima: **one optimization**.

Lowering the cross-entropy **loss function** lowers the KL divergence by exactly the same amount; every training step pushes q_θ toward p .

Back to the Waiter's Code, one last time

You now own the full toolkit: entropy, cross-entropy, KL.

Revisit the demo's *distributions* with new eyes.



Different menus, different beliefs: when the code is built for the **wrong** distribution, every wasted bit you see *is* a KL divergence.

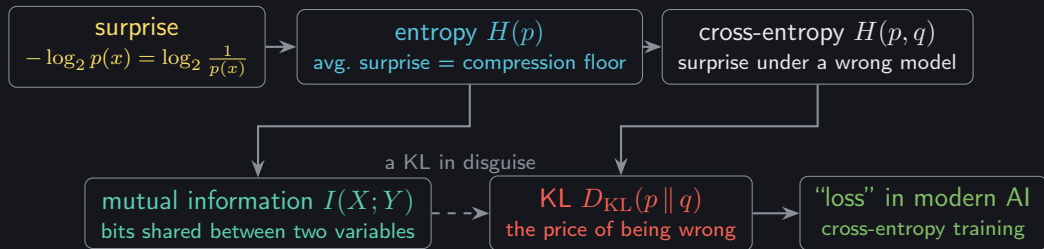
Final remarks



Prediction and compression are **mathematically equivalent**:
a model's cross-entropy loss *is* its compression rate.

A model that predicts language well *is* an optimal compressor of it:
compressibility and intelligence go hand in hand.

Takeaways to remember



surprise $\rightarrow$ entropy $\rightarrow$ cross-entropy $\rightarrow$ KL $\rightarrow$ modern AI, and the bits variables **share**.

Questions?

surprise → entropy → cross-entropy → KL → modern AI